Name:- JOGANI KRISH

Software Testing Assignment

Module- 1(Fundamental)

1. What is SDLC?

➔ SDLC is a structure imposed on the development of a software product that defines the process for planning, implementation, testing,documentation, deployment, and ongoing maintenance and support. There are a number of different development models.

➔ A Software Development Life Cycle is essentially a series of steps, or phases, that provide a model for the development and lifecycle management of an application or piece of software.

2. What is Software Testing?

➔ ‘The process consisting of all life cycle activities, both static and dynamic, concerned with planning, preparation and evaluation of software products and related work products to determine that they satisfy requirements, to demonstrate that they are fit for purpose and to detect defects.’

➔ Software Testing is a process used to identify the correctness, completeness, and quality of developed computer software.

3. What is Agile Methodology?

➔ Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product.

➔ Agile Methods break the product into small incremental builds.These builds are provided in iterations.Each iteration typically lasts from about

one to three weeks.Every iteration involves cross functional teams working simultaneously on various areas like planning, requirements analysis,design, coding, unit testing, and acceptance testing.

4. What is SRS?

➔ A software requirements specification (SRS) is a complete description of the behavior of the system to be developed. It includes a set of use cases that describe all of the interactions that the users will have with the software.Use cases are also known as functional requirements. In addition to use cases, the SRS also contains nonfunctional requirements.

➔ Non-functional requirements are requirements which impose constraints on the design or implementation (such as performance requirements, quality standards, or design constraints). Recommended approaches for the specification of software requirements are described by IEEE 830-1998.

5. What is OOPS?

➔ Identifying objects and assigning responsibility to these objects. Objects communicate to other objects by sending messages. Messages are received by the methods of an object An object is like a black box. The internal details are hidden.

➔ Object is derived from abstract data type Object-oriented programming has a web of interacting objects, each house-keeping its own state. Objects of a program interact by sending messages to each other.

6. Write basic concepts of oops?

➔ Object
Class
Encapsulation
Inheritance
Polymorphism
- Overriding
- Overloading and Abstraction

7. What is Object?

➔ An object represents an individual, identifiable item, unit, or entity,either real or abstract, with well-defined role in the problem domain. An "object" is anything to which a concept applies.

➔ This is the basic unit of object oriented programming(OOP). That is both data and function that operate on data are bundled as a unit called as object.

8. What is Class?

➔ When you define a class, you define a blueprint for an object. This doesn't actually define any data, but it does define what the class name means, that is, what an object of the class will consist of and what operations can be performed on such an object.

➔ A class represents an abstraction of the object and abstracts the properties and behavior

of that object. Class can be considered as the blueprint or definition or a template for an object and describes the properties and behavior of that object, but without any actual existence.

9. What is Encapsulation?

➔ Encapsulation is the practice of including in an object everything it needs hidden from other objects. The internal state is usually not accessible by other objects.

➔ Encapsulation in Java is the process of wrapping up of data (properties) and behavior (methods) of an object into a single unit; and the unit here is a Class (or interface). Encapsulate in plain English means to enclose or be enclosed in or as if in a capsule. In Java, a class is the capsule (or unit).

10. What is Inheritance?

➔ Inheritance means that one class inherits the characteristics of another class. This is also called a “is a” relationship.

➔ One of the most useful aspects of object-oriented programming is code reusability. As the name suggests Inheritance is the process of forming a new class from an existing class that is from the existing class called as base class, new class is formed called as derived class.

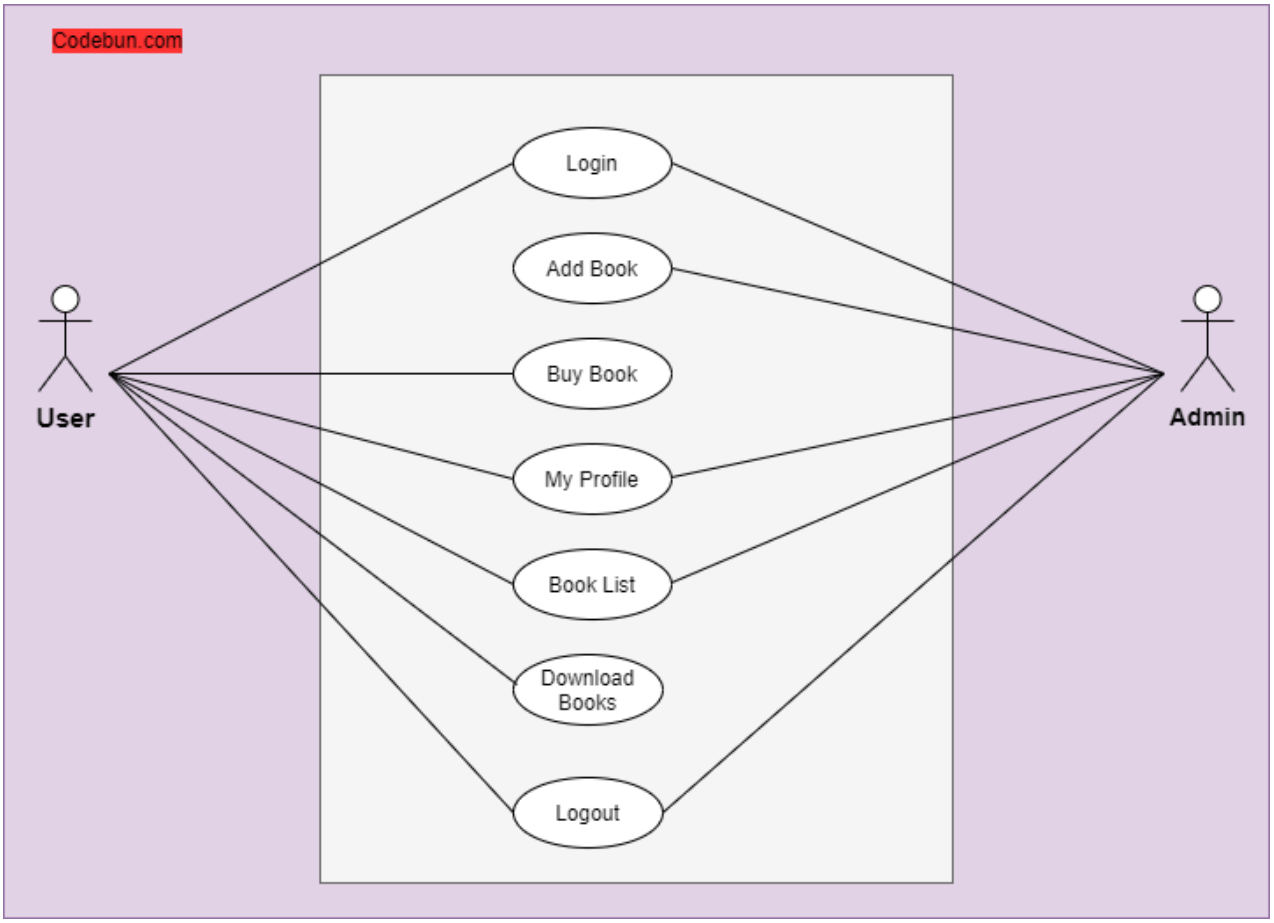
11. What is Polymorphism?

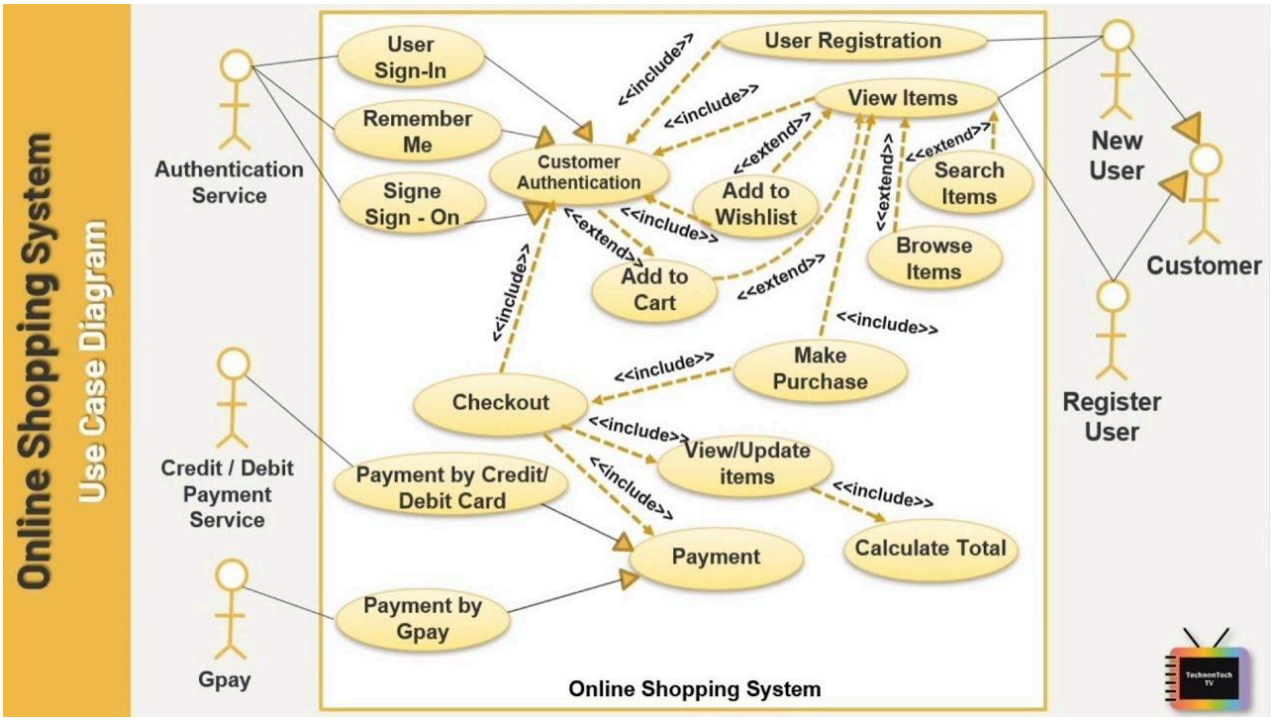
➔ Polymorphism means “having many forms”. It allows different objects to respond to the same message in different ways, the response specific to the type of the object.

➔ The most important aspect of an object is its behaviour. A behaviour is initiated by sending a message to the object. The ability to use an operator or function in different ways in other words giving different meaning or functions to the operators or functions is called polymorphism.

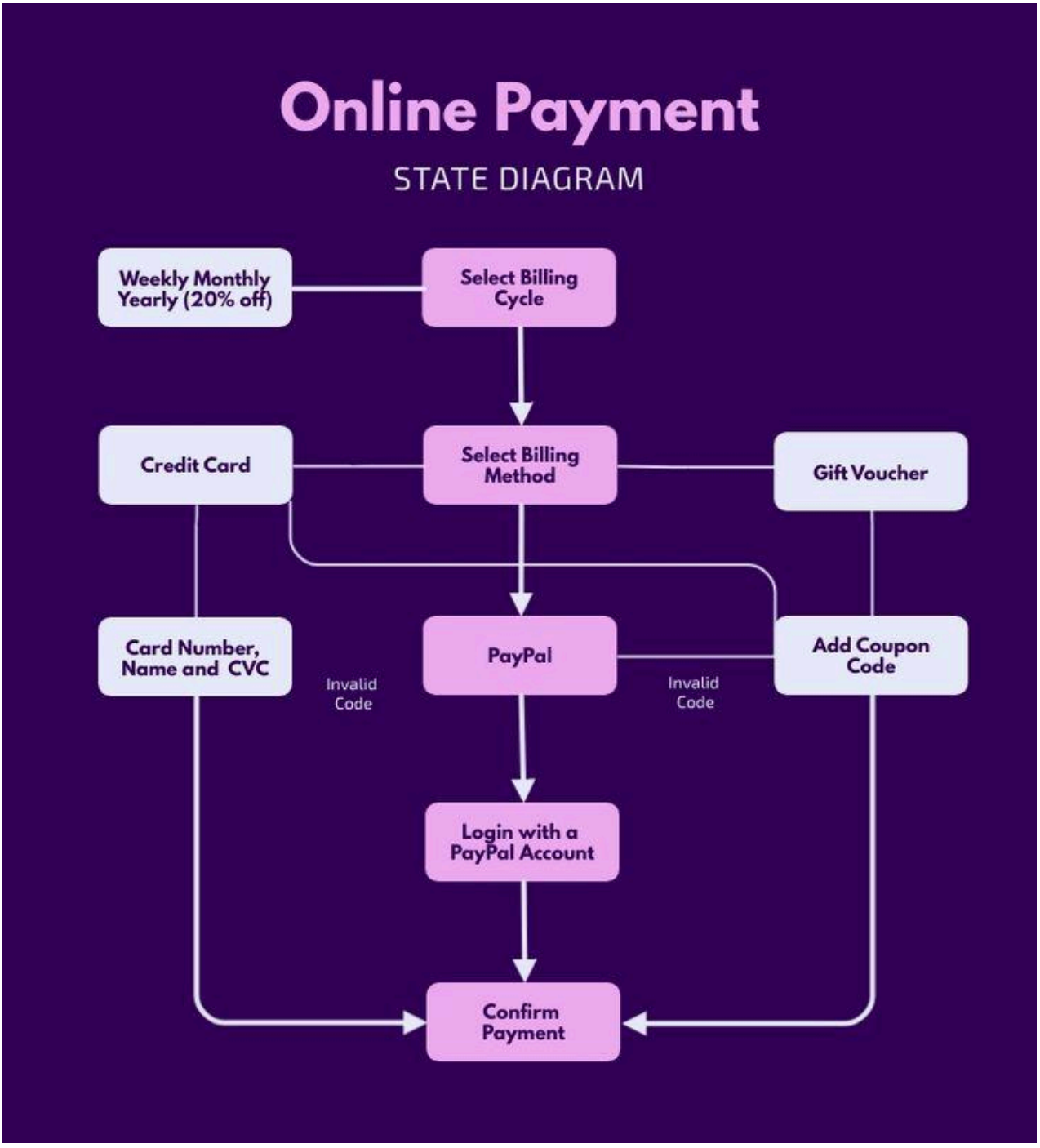
➔ There is two types of polymorphism in Java Compile time polymorphism(Overloading) and Runtime polymorphism(Overriding).

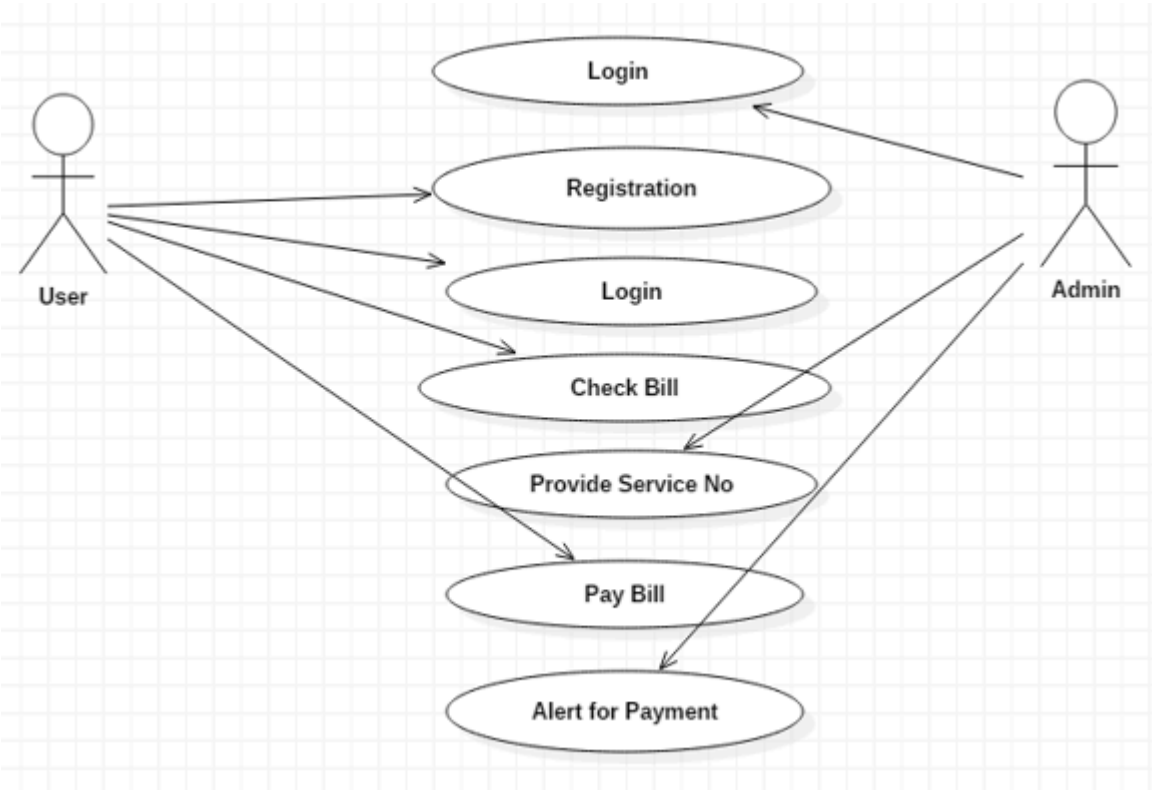
12. Draw Usecase on Online book shopping?

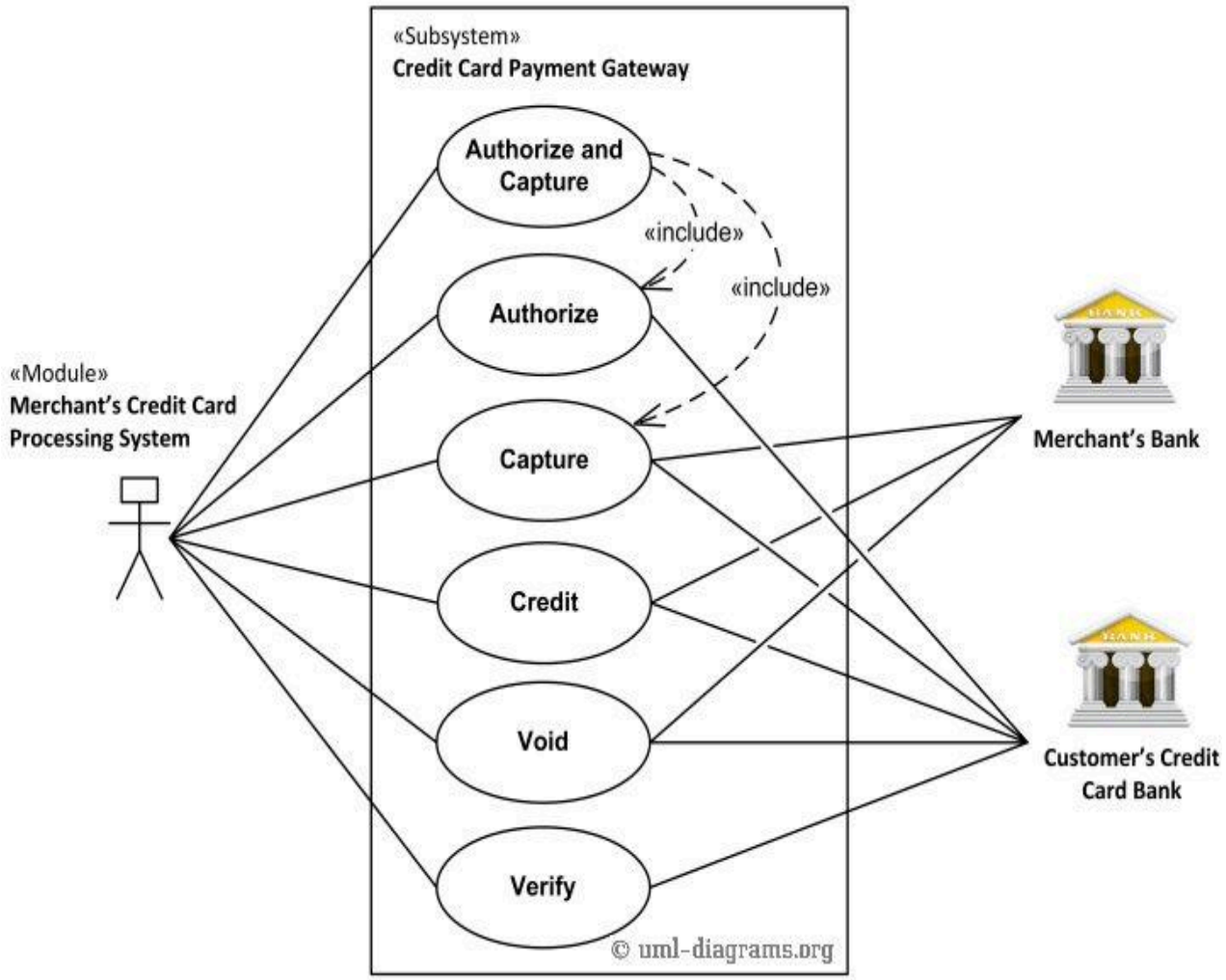




13. Draw Usecase on Online bill payment system?




Login
Registration
Login
Check Bill
Provide Service No
Pay Bill
Alert for Payment
User
Admin


«Subsystem»
Credit Card Payment Gateway
Authorize and Capture
«include»
«include»
Authorize
Capture
Credit
Void
Verify
© uml-diagrams.org
«Module»
Merchant's Credit Card Processing System
Merchant's Bank
Customer's Credit Card Bank

14. Write SDLC phases with basic introduction?

➔ SDLC is a structure imposed on the development of a software product that defines the process for planning, implementation, testing, documentation, deployment, and ongoing maintenance and support.

➔ There are a number of different development models. A Software Development Life Cycle is essentially a series of steps, or phases, that provide a model for the development and lifecycle management of an application or piece of software.

➔ There are 6 Phases in SDLC:-

1. **Requirements Collection/Gathering**
2. **Analysis**
3. **Design**
4. **Implementation**
5. **Testing**
6. **Maintenance**

1. Requirements Collection/Gathering:-

➔ Requirements definitions usually consist of natural language, supplemented by (e.g., UML) diagrams and tables.

➔ Three types of problems can arise: Lack of clarity: It is hard to write documents that are both precise and easy-to-read. Requirements confusion: Functional and Non-functional requirements tend to be intertwined. Requirements Amalgamation: Several different requirements may be expressed together.

2. Analysis:-

➔ The analysis phase defines the requirements of the system, independent of how these requirements will be accomplished. This phase defines the problem that the customer is trying to solve. The deliverable result at the end of this phase is a requirement document.

3. Design:-

➔ Design Architecture Document, Implementation Plan, Critical Priority Analysis, Performance Analysis, Test Plan. The Design team can now expand upon the information established in the requirement document. The requirement document must guide this decision process.

4. Implementation:-

➔ In the implementation phase, the team builds the components either from scratch or by composition. Given the architecture document from the design phase and the requirement document from the analysis phase, the team should build exactly what has been requested, though there is still room for innovation and flexibility.

5. Testing:-

➔ Simply stated, quality is very important. Many companies have not learned that

quality is important and deliver more claimed functionality but at a lower quality level.

➔ Regression Testing, Internal Testing, Unit Testing, Application Testing, Stress Testing.

6. Maintenance:-

➔ Software maintenance is one of the activities in software engineering, and is the process of enhancing and optimizing deployed software (software release), as well as fixing defects.

➔ Software maintenance is also one of the phases in the System Development Life Cycle (SDLC), as it applies to software development. The maintenance phase is the phase which comes after deployment of the software into the field.

➔ Corrective maintenance: identifying and repairing defects.

Adaptive maintenance: adapting the existing solution to the new platforms.

Perfective Maintenance: implementing the new requirements.

15. Explain Phases of the Waterfall Model?

➔ Requirements are very well documented, clear and fixed. Product definition is stable. Technology is understood and is not dynamic. There are no ambiguous requirements. Ample resources with required expertise are available to support the product. The project is short.

➔ The waterfall model is a Software Development Model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to Project Management and Software Development.

➔ The Waterfall Model is useful in situations where the project requirements are well-defined and the project goals are clear. It is often used for large-scale projects with long timelines, where there is little room for error

and the project stakeholders need to have a high level of confidence in the outcome.

Features of Waterfall Model

Following are the features of the waterfall model:

1. **Sequential Approach**: The waterfall model involves a sequential approach to software development, where each phase of the project is completed before moving on to the next one.
2. **Document-Driven:** The waterfall model depended on documentation to ensure that the project is well-defined and the project team is working towards a clear set of goals.
3. **Quality Control:** The waterfall model places a high emphasis on quality control and testing at each phase of the project, to

ensure that the final product meets the requirements and expectations of the stakeholders.

4. **Rigorous Planning**: The waterfall model involves a careful planning process, where the project scope, timelines, and deliverables are carefully defined and monitored throughout the project lifecycle.

Importance of Waterfall Model

Following are the importance of waterfall model:

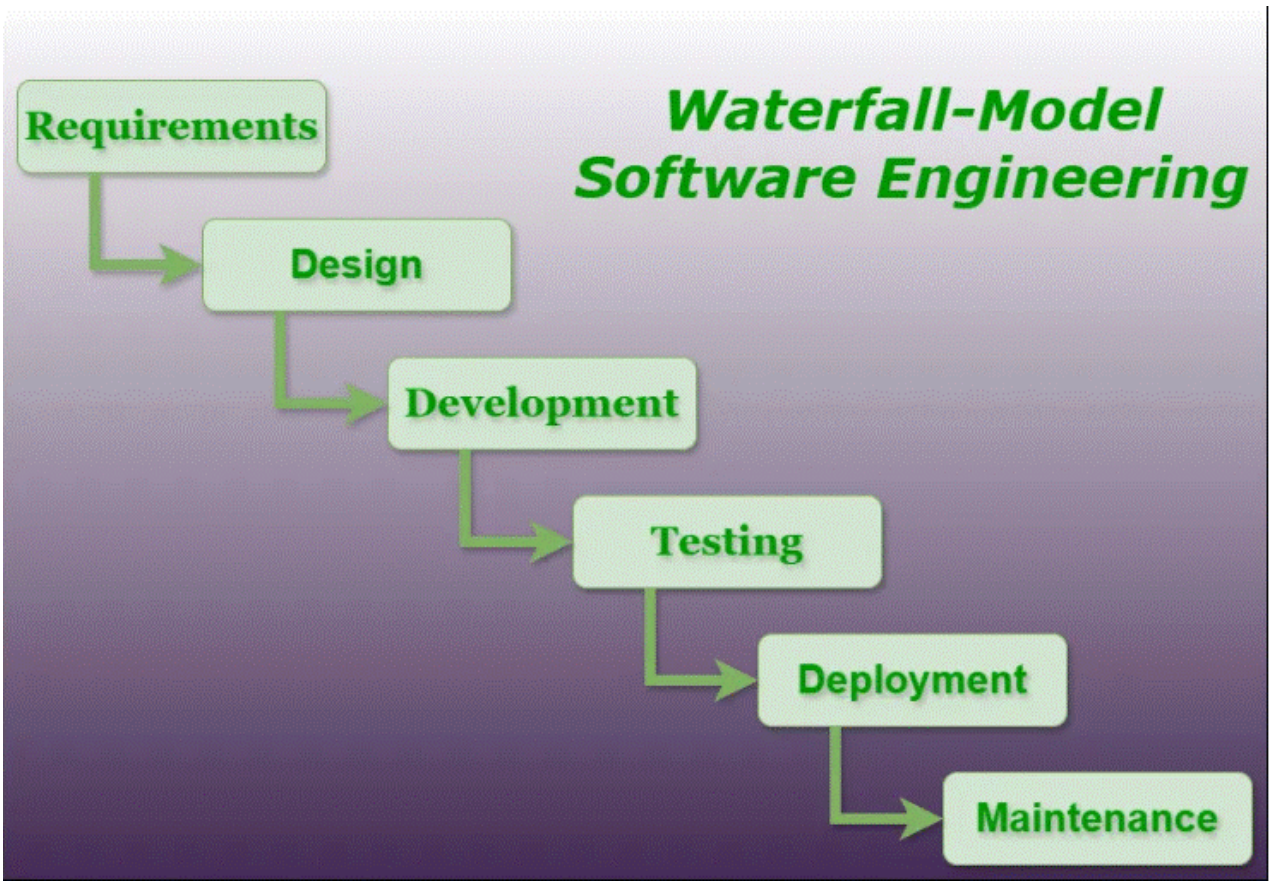
1. **Clarity and Simplicity:** The linear form of the Waterfall Model offers a simple and unambiguous foundation for project development.
2. **Clearly Defined Phases:** The Waterfall Model phases each have unique inputs and

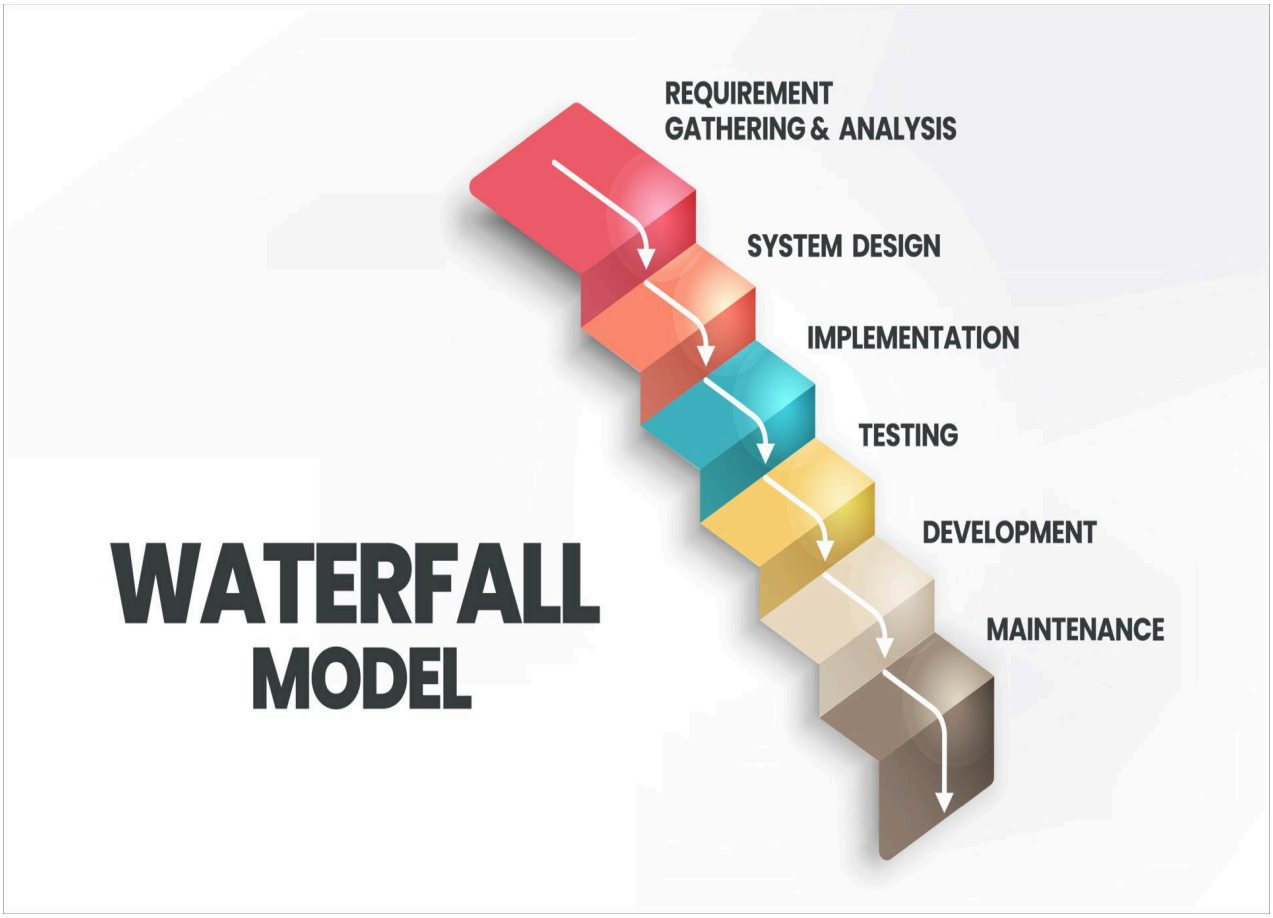
outputs, guaranteeing a planned development with obvious checkpoints.

3. **Documentation:** A focus on thorough documentation helps with software comprehension, maintenance, and future growth.
4. **Stability in Requirements:** Suitable for projects when the requirements are clear and stable, reducing modifications as the project progresses.
5. **Resource Optimization:** It encourages effective task-focused work without continuously changing contexts by allocating resources according to project phases.

6. **Relevance for Small Projects:**

   Economical for modest projects with simple specifications and minimal complexity.


Waterfall-Model
Software Engineering
Requirements
Design
Development
Testing
Deployment
Maintenance


WATERFALL
MODEL
REQUIREMENT
GATHERING & ANALYSIS
SYSTEM DESIGN
IMPLEMENTATION
TESTING
DEVELOPMENT
MAINTENANCE

1. Requirement Gathering and Analysis: Firstly all the requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed.

The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (an inconsistent requirement is one in which some part of the requirement contradicts some other part).

2. Requirement Specification: These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

2. Design

The goal of this **Software Design Phase** is to convert the requirements acquired in the SRS

into a format that can be coded in a programming language. It includes high-level and detailed design as well as the overall software architecture. A **Software Design Document** is used to document all of this effort (SDD).

- **High-Level Design (HLD)**: This phase focuses on outlining the broad structure of the system. It highlights the key components and how they interact with each other, giving a clear overview of the system's architecture.
- **Low-Level Design (LLD)**: Once the high-level design is in place, this phase zooms into the details. It breaks down each component into smaller parts and provides specifics about how each part will function, guiding the actual coding process.

3. Development

In the **Development Phase** software design is translated into source code using any suitable programming language. Thus each designed module is coded. The unit testing phase aims to check whether each module is working properly or not.

- In this phase, developers begin writing the actual source code based on the designs created earlier.
- The goal is to transform the design into working code using the most suitable programming languages.
- Unit tests are often performed during this phase to make sure that each component functions correctly on its own.

4. Testing and Deployment

1. Testing: Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is

carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists of three different kinds of testing activities as described below.

- **Alpha testing:** Alpha testing is the system testing performed by the development team.
- **Beta testing:** Beta testing is the system testing performed by a friendly set of customers.
- **Acceptance testing:** After the software has been delivered, the customer performs acceptance testing to determine whether to accept the delivered software or reject it.

2. Deployment: Once the software has been thoroughly tested, it's time to deploy it to the

customer or end-users. This means making the software ready and available for use, often by moving it to a live or staging environment.

During this phase, we also focus on helping users get comfortable with the software by providing training, setting up necessary environments, and ensuring everything is running smoothly. The goal is to make sure the system works as expected in real-world conditions and that users can start using it without any hitches.

5. Maintenance

In **Maintenance Phase** is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop a full software. There are three types of maintenance.

- **Corrective Maintenance:** This type of maintenance is carried out to correct errors that were not discovered during the product development phase.

- **Perfective Maintenance:** This type of maintenance is carried out to enhance the functionalities of the system based on the customer's request.
- **Adaptive Maintenance:** Adaptive maintenance is usually required for porting the software to work in a new environment such as working on a new computer platform or with a new operating system.

16. Write Phases of Spiral Model.

➔ Spiral Model is very widely used in the software industry as it is in synch with the natural development process of any product i.e. learning with maturity and also involves minimum risk for the customer as well as the development firms.

➔ When costs there are a budget constraint and risk evaluation is important. For medium

to high-risk projects. Long-term project commitment because of potential changes to economic priorities as the requirements change with time.

➔ Customer is not sure of their requirements which are usually the case. Requirements are complex and need evaluation to get clarity.

Spiral Model Phases

Phases of the Spiral Model

The Spiral Model is a risk-driven model, meaning that the focus is on managing risk through multiple iterations of the software development process. **Each phase of the Spiral Model is divided into four Quadrants:**

1. Objectives Defined

In first phase of the spiral model we clarify what the project aims to achieve, including functional and non-functional requirements.

Requirements are gathered from the customers and the objectives are identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.

2. Risk Analysis and Resolving

In the risk analysis phase, the risks associated with the project are identified and evaluated.

During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.

3. Develop the next version of the Product

During the third quadrant, the identified features are developed and verified through testing. At the

end of the third quadrant, the next version of the software is available.

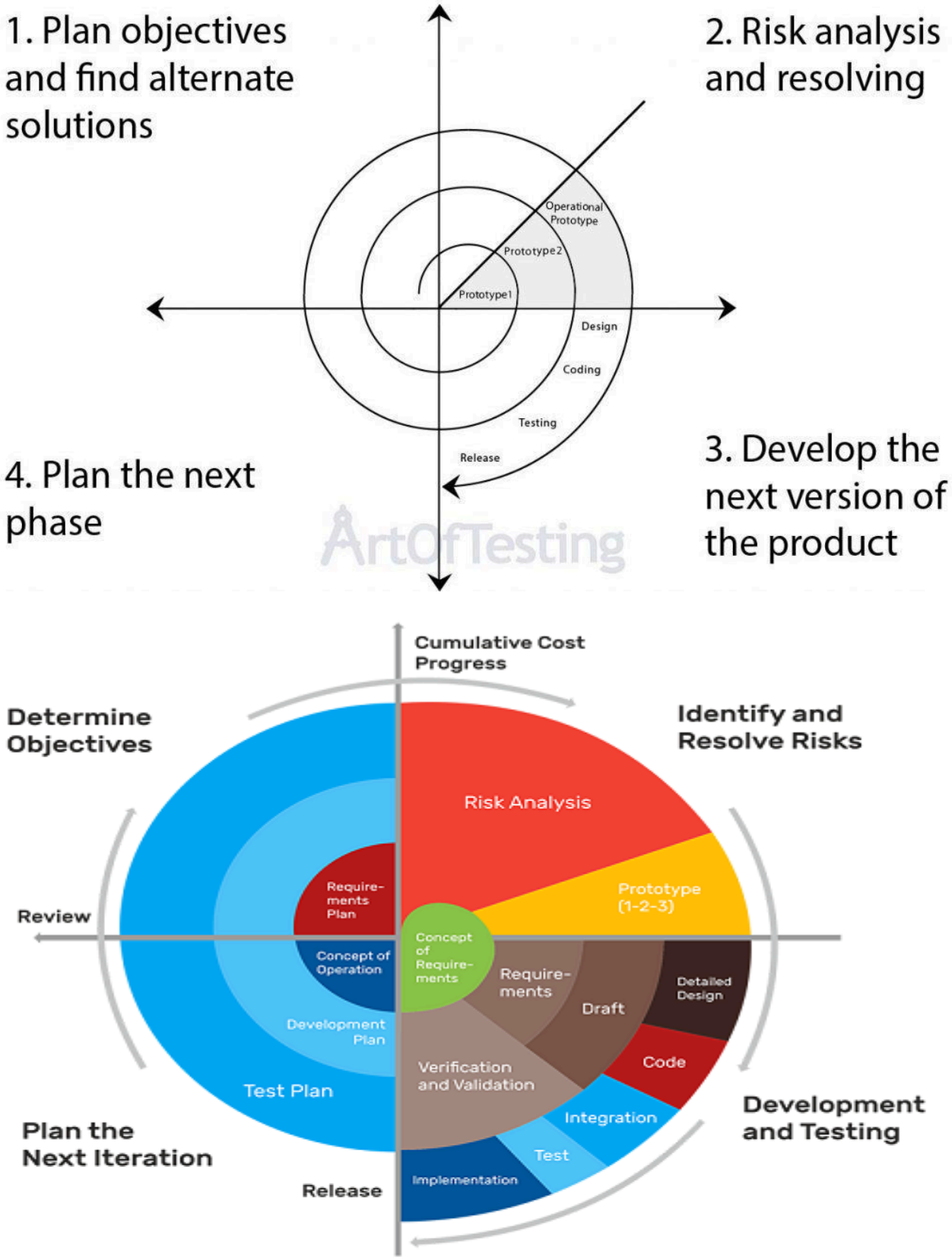
In the evaluation phase, the software is evaluated to determine if it meets the customer's requirements and if it is of high quality.

4. Review and plan for the next Phase

In the fourth quadrant, the Customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.

The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.

The Spiral Model is often used for complex and large software development projects, as it allows for a more flexible and adaptable approach to **Software development**. It is also well-suited to projects with significant uncertainty or high levels of risk.



Spiral Model Methodology and its Phases

17. Write Agile manifesto principles.

➔ Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product. Agile Methods break the product into small incremental builds.

12 Principles of Agile Manifesto

1. **Customer Satisfaction through Early and Continuous Delivery:** This principle

concentrates on the importance of customer satisfaction by providing information to customers early on time and also with consistency throughout the development process.

2. **Welcome Changing Requirements, Even Late in Development:** Agile processes tackle change for the customer's competitive advantage. Even late in development, changes in requirements are welcomed to ensure the delivered software meets the evolving requirements of the customer.
3. **Deliver Working Software Frequently:** This principle encourages the regular release of functional software increments in short iterations. This enables faster

feedback and adaptation to changing requirements.

4. **Collaboration between Business Stakeholders and Developers:** This says the businesspeople and developers must work together daily throughout the project. There should be communication and collaboration between stakeholders and the development team regularly. This is crucial for understanding and prioritizing requirements effectively.
5. **Build Projects around Motivated Individuals**: This promotes in giving developers the environment and support they need and trusts them to complete the job successfully. Motivated and empowered individuals are more likely to produce work

with quality and make valuable contributions to the project.

6. **Face-to-face communication is the Most Effective:** Face-to-face communication is the most effective method of discussion and conveying information. This principle depicts the importance of direct interaction which helps minimize misunderstandings, and hence effective communication is achieved.
7. **Working Software is the Primary Measure of Progress:** This principle emphasizes delivering functional and working software as the primary metric for project advancement. It encourages teams to prioritize the continuous delivery of valuable features, so it ensures that good

progress is consistently achieved throughout the process. The primary goal is to provide customers with incremental value and also gather feedback early in the project life cycle.

8. **Maintain a Sustainable Pace of Work:**

   Agile promotes sustainable development. All people involved: The sponsors, developers, and users should be able to maintain a constant pace indefinitely. This principle depicts the need for a sustainable and consistent development pace. This helps in avoiding burnout and ensures long-term project success.

9. **Continuous Attention to Technical Excellence and Good design:** This principle is on the importance of maintaining

high standards of technical craft and design, so it ensures the long-term ability in maintenance and adaptability of the software.

10. **Simplicity—the Art of Maximizing the Amount of Work Not Done:** Simplicity is essential. The objective here is to concentrate on the most valuable features and tasks and avoid unnecessary complexity as the art of maximizing the amount of work not done is crucial.
11. **Self-Organizing Teams:** Self-organizing teams provide the best architectures, requirements, and designs. These help in empowering teams to make decisions and organize to optimize efficiency and creativity.

12. **Regular Reflection on Team**

Effectiveness: This makes the team reflect on how to become more effective at regular intervals and then adjust accordingly. Continuous improvement is very crucial for adapting to changing circumstances and optimizing the team's performance over time.

18. Explain working methodology of agile model and also write pros and cons.

➔ Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product. Agile Methods break the product into small incremental builds. These builds are provided in iterations.

➔ Each iteration typically lasts from about one to three weeks. Every iteration involves cross functional teams working simultaneously on various areas like planning, requirements analysis, design, coding, unit testing, and acceptance testing.

➔ Agile model believes that every project needs to be handled differently and the existing methods need to be tailored to best suit the project requirements. In agile the tasks are divided to time boxes (small time frames) to deliver specific features for a release.

➔ Iterative approach is taken and working software build is delivered after each iteration. Each build is incremental in terms of features; the f inal build holds all the features required by the customer. Agile thought process had started early in the software development and started becoming popular with time due to its flexibility and adaptability.

Pros

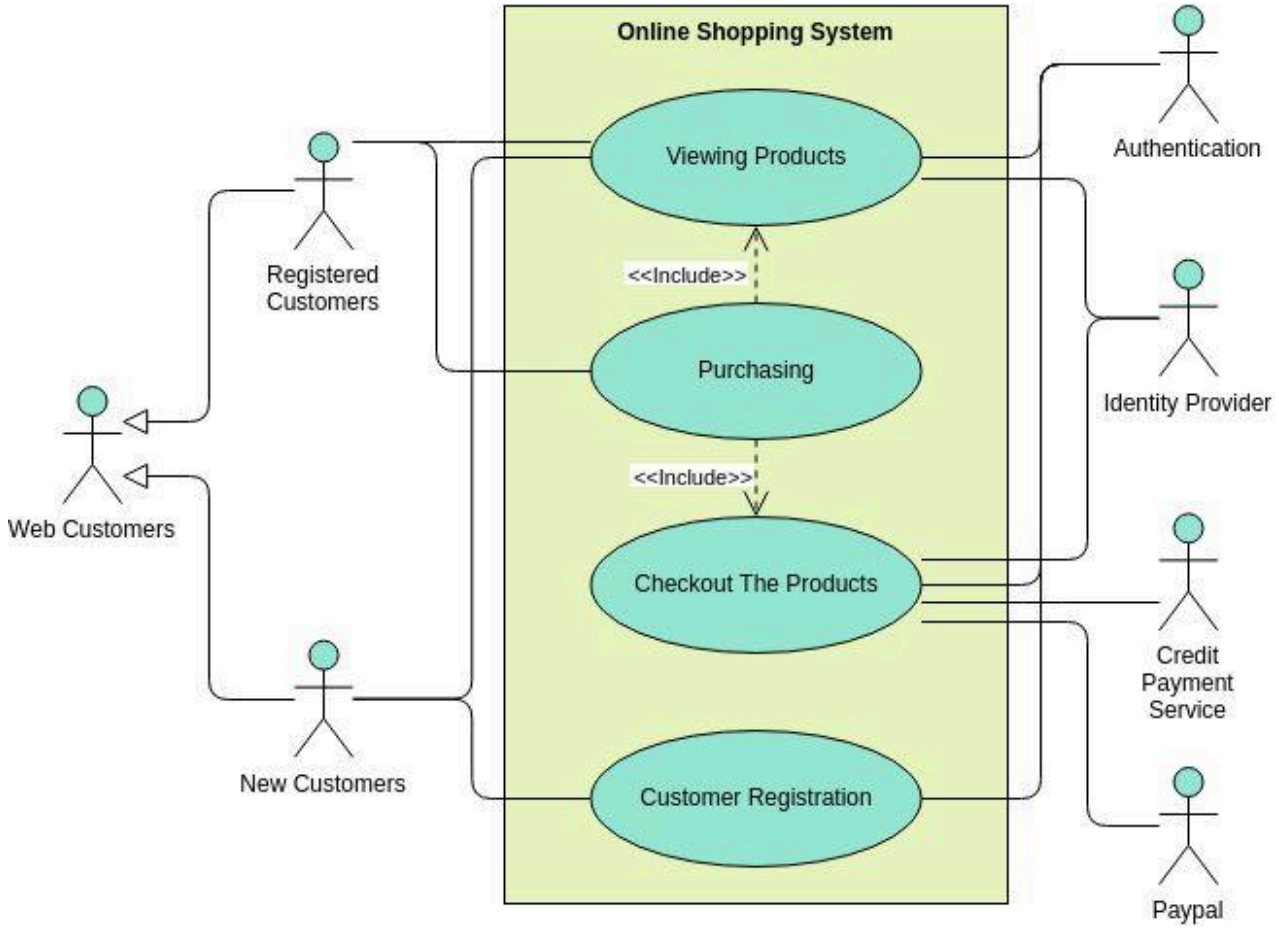
- Is a very realistic approach to software development
- Promotes teamwork and cross training. Functionality can be developed rapidly and demonstrated.
- Resource requirements are minimum.
- Suitable for fixed or changing requirements.
- Delivers early partial working solutions.
- Good model for environments that change steadily.
- Minimal rules, documentation easily employed.
- Enables concurrent development and delivery within an overall planned context.
- Little or no planning required.
- Easy to manage.
- Gives flexibility to developers.

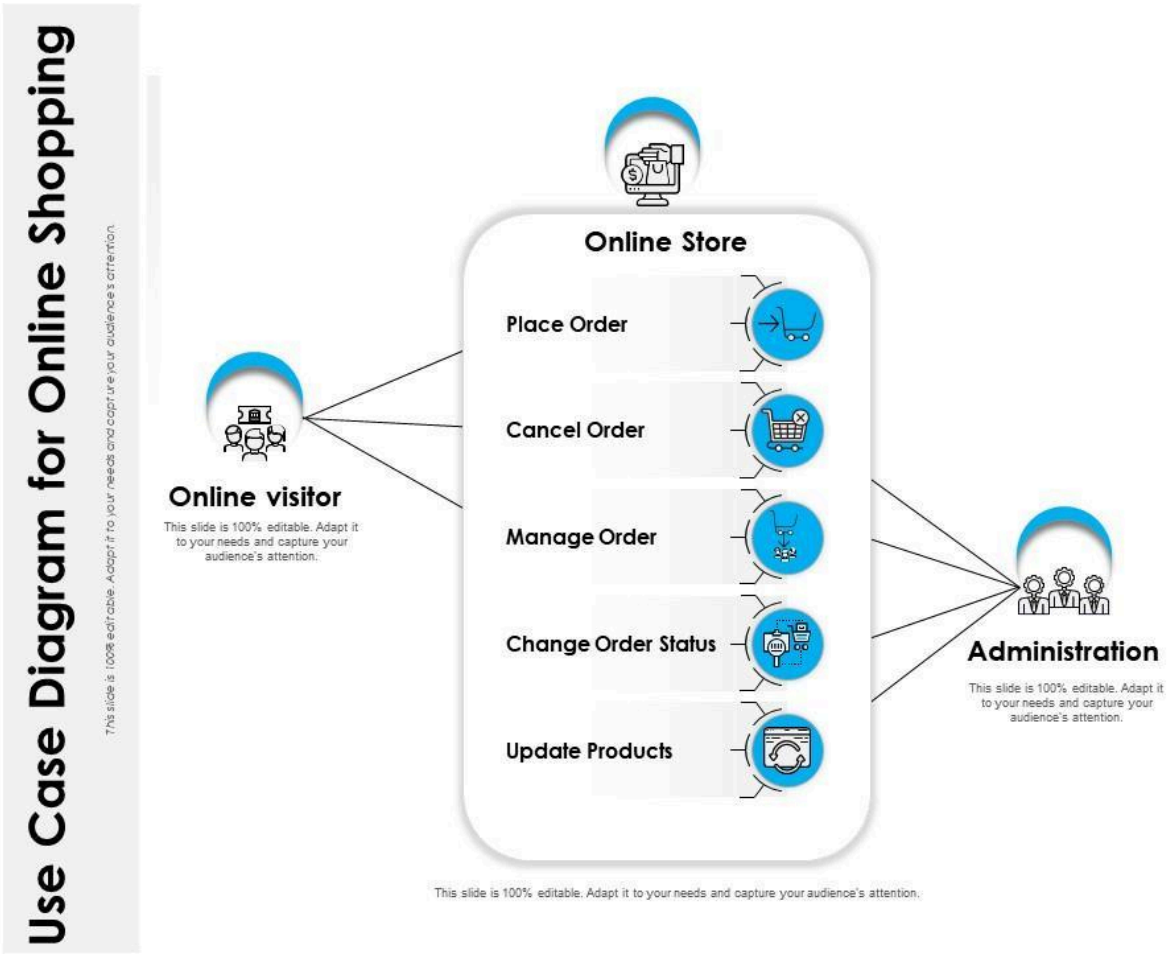
Cons

- Not suitable for handling complex dependencies.

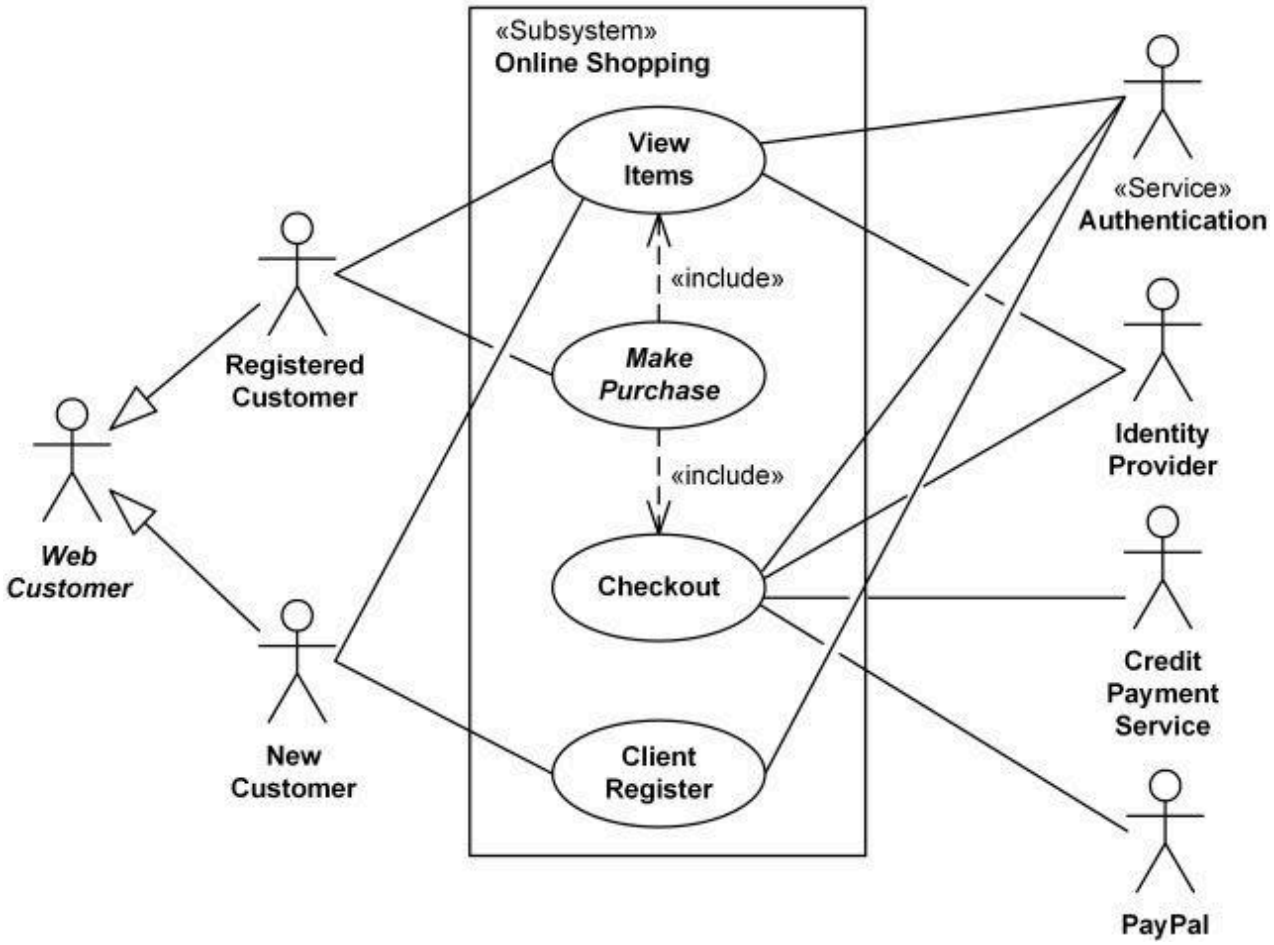
- More risk of sustainability, maintainability and extensibility.
- An overall plan, an agile leader and agile PM practice is a must without which it will not work.
- Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.
- Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.
- There is very high individual dependency, since there is minimum documentation generated.
- Transfer of technology to new team members may be quite challenging due to lack of documentation.

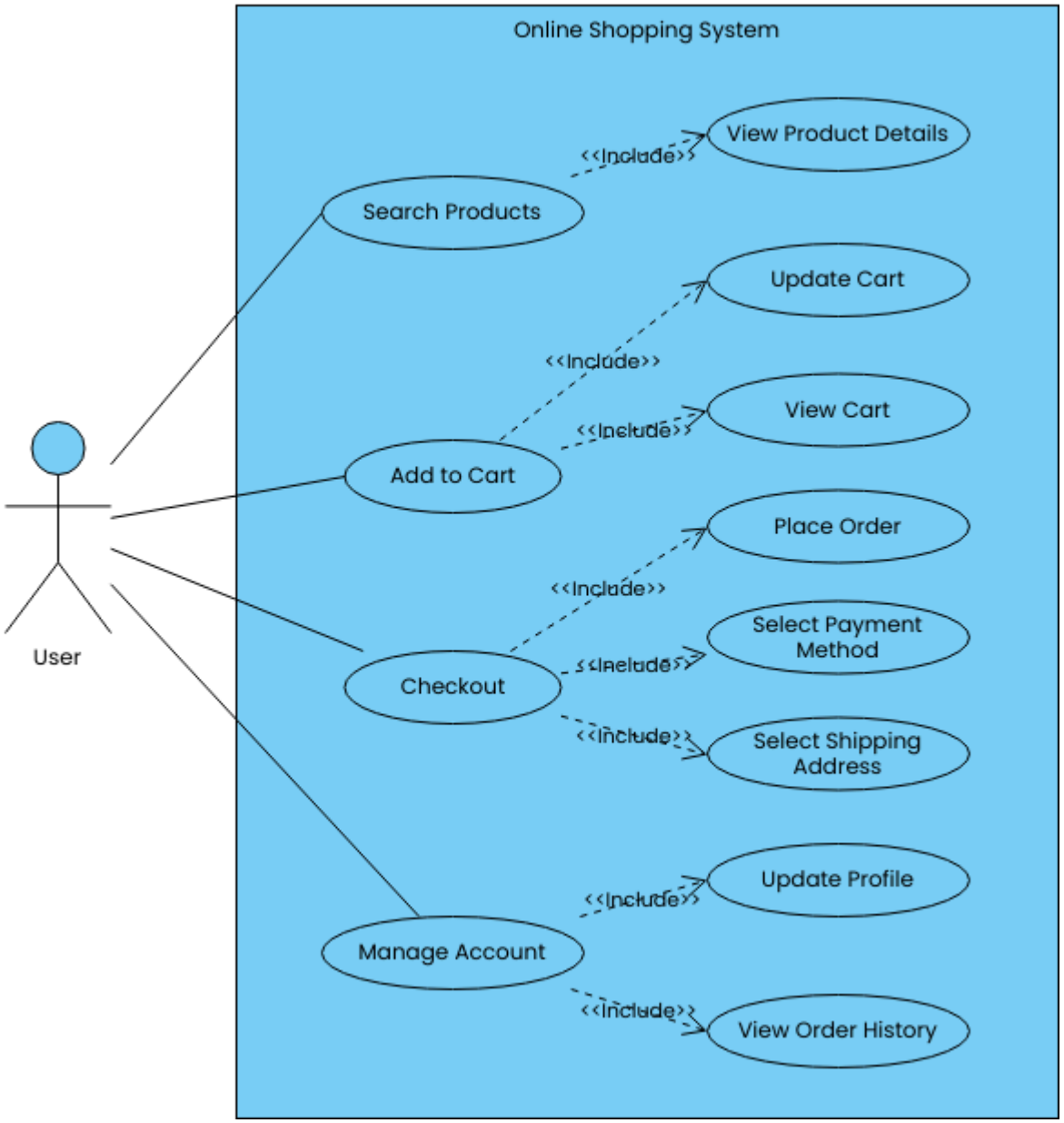
19. Draw Usecase on Online Shopping Product using COD.


Online Shopping System
Viewing Products
Purchasing
Checkout The Products
Customer Registration
<<Include>>
<<Include>>
Registered Customers
Web Customers
New Customers
Authentication
Identity Provider
Credit Payment Service
Paypal



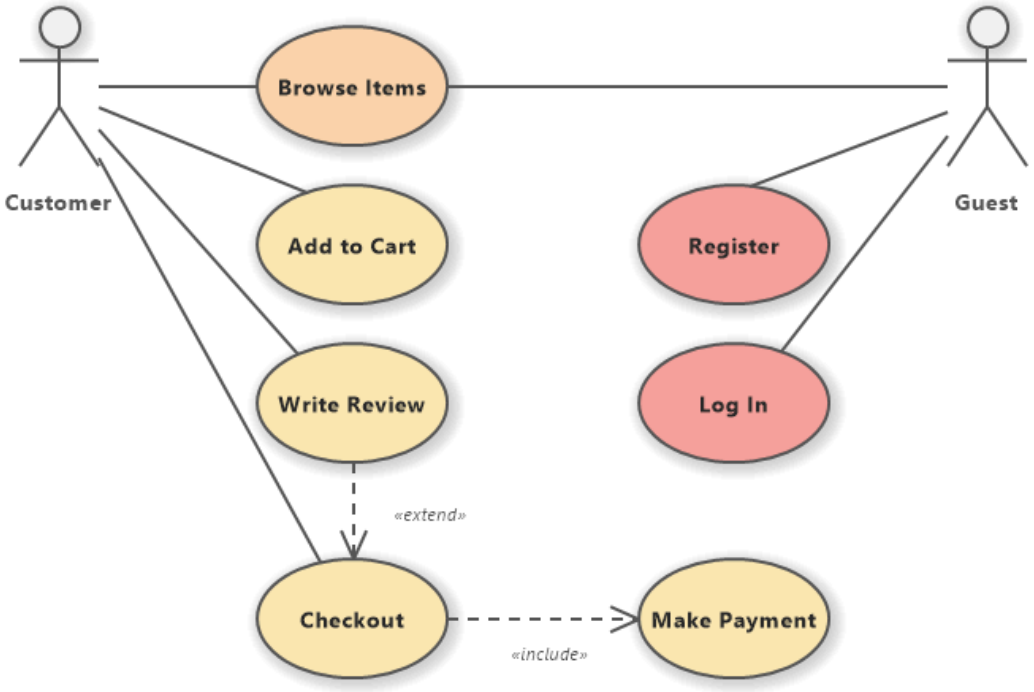
20. Draw Usecase on Online Shopping Product using payment gateway.

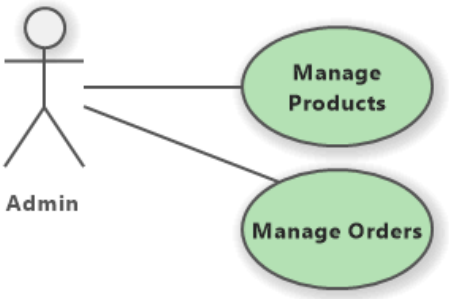

«Subsystem»
Online Shopping
View Items
«include»
Make Purchase
«include»
Checkout
Client Register
Registered Customer
Web Customer
New Customer
«Service»
Authentication
Identity Provider
Credit Payment Service
PayPal

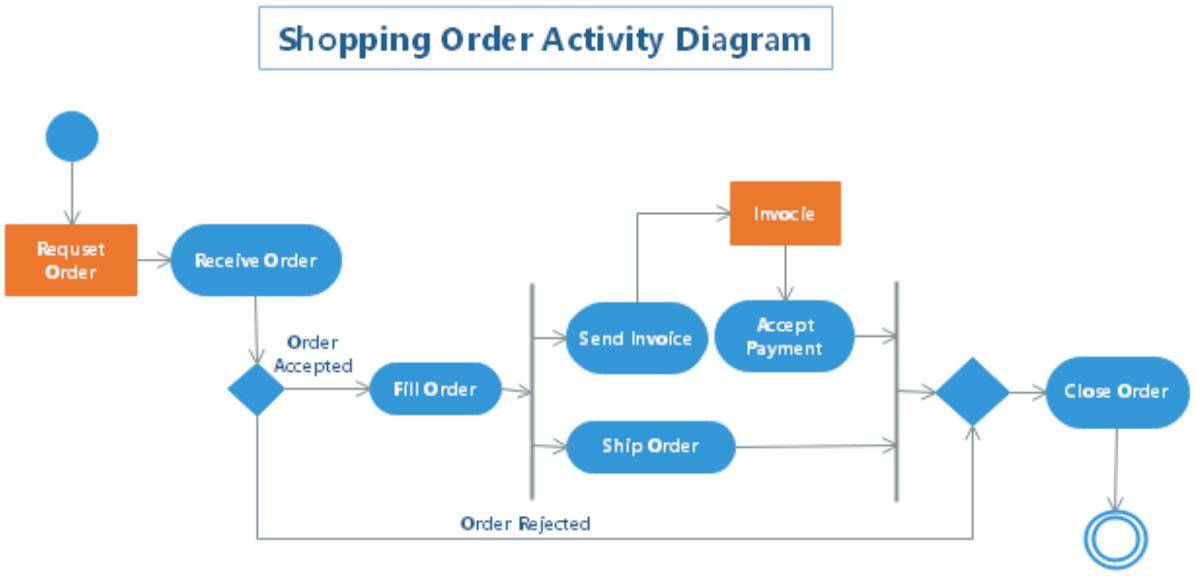

Online Shopping System
User
Search Products
Add to Cart
Checkout
Manage Account
View Product Details
Update Cart
View Cart
Place Order
Select Payment Method
Select Shipping Address
Update Profile
View Order History
<<Include>>
<<Include>>
<<Include>>
<<Include>>
<<Include>>
<<Include>>
<<Include>>
<<Include>>

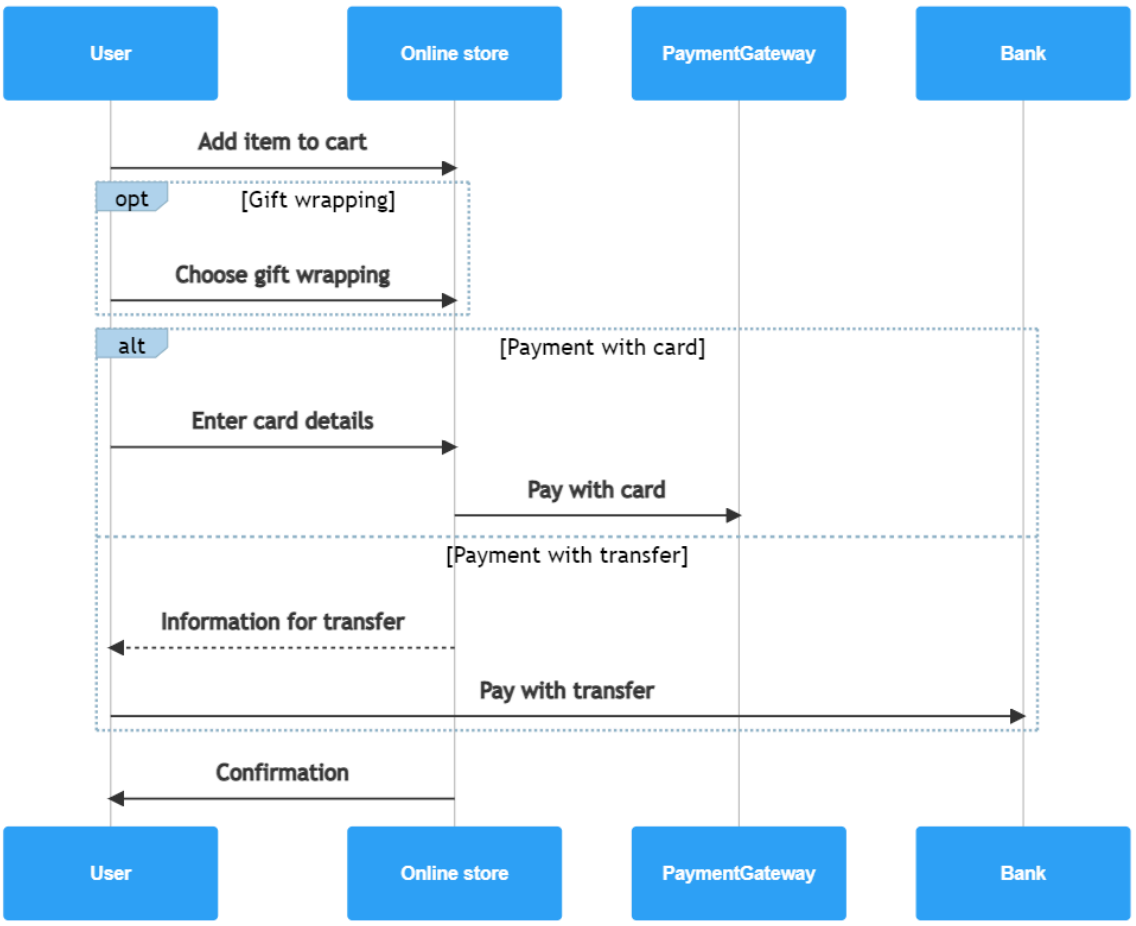
ONLINE SHOPPING SYSTEM

UML USE CASE DIAGRAM






Shopping Order Activity Diagram
Requset Order
Receive Order
Order Accepted
Fill Order
Invocie
Send Invoice
Accept Payment
Ship Order
Close Order
Order Rejected


User
Online store
PaymentGateway
Bank
Add item to cart
opt
[Gift wrapping]
Choose gift wrapping
alt
[Payment with card]
Enter card details
Pay with card
[Payment with transfer]
Information for transfer
Pay with transfer
Confirmation
User
Online store
PaymentGateway
Bank